

南宁家政 AI 动态定价算法模型报告

版本: 4.1 | 日期: 2026-05-22 | 对应模块: core/pricing_engine.py

一、模型架构：四层混合定价模型

采用**成本兜底+市场锚定+AI动态调价+规则校验**四层架构，确保定价在安全边际内实现利润最大化。

输入(订单信息)

|

▼

| 第一层：成本兜底层 (Cost Floor) |
| 复用 CostCalculator.calculate_cost() |
| 输出：保本价 $\times 1.1$ = 安全底价 |
| 约束：最终定价 $\geq$ 安全底价 (绝对硬下限) |

|

▼

| 第二层：市场锚定层 (Market Anchor) |
| 数据源：competitive_prices.csv |
| 按(城区, 服务类型, 服务等级)聚合竞品价格区间 |
| 输出：[竞品P25, 竞品P75] 参考区间 |
| 约束：最终定价 $\leq$ 竞品均价 $\times 1.3$ |

|

▼

| 第三层：AI 动态调价层 (XGBoost) |
| 25维特征 $\rightarrow$ XGBoost Regressor |
| 预测：最优定价倍率 = 建议价 / 保本价 |
| 输出：AI建议加价率 (如 1.45x) |

|

▼

| 第四层：规则校验层 (Rule Validation) |
| 区域加价(从config.yaml读取) + 季节加价 + 供需加价 |
| 最低毛利率 $\geq$ 差异化约束 (见下表) |
| 最终定价 = $\max$ (规则价, AI价, 安全底价) |

|



输出(最终售价 + 加价原因列表)

1.1 与旧模型的关键区别

维度	旧模型 (Phase 2)	新模型 (Phase 3)	AI模型 (Phase 4)
定价方法	标准价格 × 0.85 倒推	四级成本正向求和	四层混合模型
成本准确度	误差大, 忽略交通/风险	精确到每笔订单	复用 CostCalculator
市场行情	未考虑	未考虑	competitive_prices 锚定
智能调价	无	无	XGBoost 预测最优倍率
规则引擎	硬编码	config.yaml 参数化	config.yaml + 自动校验
亏损率	46.84%	0% (强制保本)	0% (安全边际)
平均毛利率	-3.45%	约20%	约38%
市场锚定乘数	—	—	1.2x → 1.3x
毛利率约束	统一25%	统一25%	服务类型差异化
区域加价	全品类	全品类	日常保洁不适用

二、特征工程

2.1 特征列表

编号	特征名	类型	说明
1-4	service_type_*	One-Hot	4种服务类型哑变量
5	duration	数值	服务时长(h)
6	skill_level	有序	新手=0, 熟练=1, 金牌=2
7	district_ordinal	有序	武鸣=0 → 青秀=6
8-14	district_*	One-Hot	7城区哑变量

编号	特征名	类型	说明
15	community_tier	有序	低端=0 → 高端=4
16	community_premium	数值	加价系数 1.0~1.15
17	month	数值	月份 1-12
18	is_weekend	二元	是否周六日
19	is_sanyuesan	二元	是否三月三期间
20	is_huinantian	二元	3-4月+深度清洁
21	is_spring_festival	二元	是否1月
22	season_level	有序	平季=0, 普通旺季=1, 超级旺季=2
23	supply_demand_ratio	数值	当月城区供需比
24	break_even_price	数值	四级成本保本价
25	labor_cost_ratio	数值	人力成本/总成本

目标变量: price_multiple = 订单金额 / 保本价（即历史成交价是保本价的多少倍）

2.2 特征重要性 (Top 15)

排名	特征	重要性	分析
1	service_type_深度清洁	40.4%	深度清洁定价差异最大
2	service_type_家电清洗	19.7%	家电清洗按台计价，差异明显
3	service_type_日常保洁	11.2%	日常保洁基准定价
4	season_level	6.0%	季节性波动影响定价
5	labor_cost_ratio	3.6%	人力占比影响利润率
6	service_type_开荒保洁	2.8%	开荒保洁按面积计价
7	supply_demand_ratio	2.6%	供需关系调节价格
8	community_premium	2.3%	小区档次对价格的影响

排名	特征	重要性	分析
9	skill_level	1.9%	师傅等级溢价
10-15	其它	<10%	月份、城区、时长等

三、模型训练

3.1 训练配置

项目	值
模型	XGBoost Regressor
训练数据	sample_orders.csv 2025年订单（7,856条）
验证数据	sample_orders.csv 2026年订单（4,876条）
特征维度	25
目标变量	最优定价倍率 (训练集均值 1.41, 验证集均值 1.52)
超参数	max_depth=5, lr=0.05, subsample=0.8, colsample=0.8
早停轮数	30 (实际训练309轮)
节假日判断	chinese-calendar 动态计算（三月三/春节按农历）

3.2 训练效果

指标	训练集	验证集
R ²	0.8505	—
MAE	0.084	0.096
RMSE	0.151	0.167
MAPE	—	7.84%

解读: 模型能解释85%的定价倍率方差，训练集MAE仅0.084（平均定价倍率偏差约8个百分点），验证集MAPE为7.84%，泛化能力良好。

四、回测结论

4.1 总体对比

对2026年4,876笔订单进行回测（使用实际订单日期，季节乘数在对应月份正确激活）：

指标	AI定价	规则定价	旧模型
亏损率	0.00%	0.00%	46.84%
平均毛利率	38.21%	37.99%	-3.45%
MAPE	30.40%	30.56%	32.71%
准确率	69.75%	69.58%	67.41%
中位数倍率(vs保本价)	1.558x	1.546x	—

关于亏损率: 0% 亏损率由 $\text{final_price} = \max(\text{floor_price}, \text{rule_price}, \text{ai_price})$ 中的 $\text{floor_price} = \text{保本价} \times 1.1$ 硬下限保证，是系统设计的数学恒等式，不作为模型质量度量。**MAPE、毛利率、中位数倍率才是有效度量指标。**

关键发现:

- 亏损率从46.84%降到0%：成本兜底层（保本价 $\times$ 1.1）彻底杜绝了亏损订单
- 毛利率从-3.45%提升到38.21%：修复了竞品数据偏低导致市场锚定过度压制价格的问题
- MAPE从32.71%降至30.40%：AI定价比旧模型更接近市场实际成交价
- 中位数倍率1.56x：AI定价典型地比保本价高出56%，处于健康盈利区间
- 市场锚定乘数从1.2x调整至1.3x：反映竞品listing价与实际成交价的自然差距

4.2 分服务类型表现

服务类型	MAPE	毛利率	偏误	特点
日常保洁	26.7%	18.9%	+20.9%	充分竞争品类，毛利率受限，禁用区域加价
深度清洁	20.7%	26.1%	-5.8%	表现最佳，回南天缺口仅-4.3%
家电清洗	37.5%	36.0%	+0.7%	偏差最小，但家电种类差异大导致MAPE偏高
开荒保洁	54.3%	29.3%	+20.9%	缺乏面积特征，按时长定价精度有限

4.3 回南天缺口修复

指标	旧模型	AI定价	改善
回南天深度清洁均价	¥287.4	¥396.3	+37.9%
与实际成交价缺口	-30.6%	-4.3%	缩小26.3pp
亏损率	52%	0%	完全修复

4.4 各城区表现

城区	AI亏损率	规则亏损率	旧模型亏损率
青秀	0.0%	0.0%	36.0%
良庆	0.0%	0.0%	39.5%
江南	0.0%	0.0%	54.9%
西乡塘	0.0%	0.0%	56.1%
兴宁	0.0%	0.0%	57.1%
邕宁	0.0%	0.0%	54.3%
武鸣	0.0%	0.0%	42.1%

五、API 端点说明

端点	方法	说明
/api/price	POST	单次定价，输入订单信息，返回定价结果+加价原因
/api/batch-price	POST	批量定价，用于前端批量试算
/api/model-info	GET	模型信息（特征重要性、版本、训练日期）
/api/backtest	POST	触发回测
/api/backtest-results	GET	获取最近回测结果

启动方式

```
python -m uvicorn backend.main:app --reload --port 8000
```

请求示例

```
POST /api/price
{
  "service_type": "日常保洁",
  "skill_level": "熟练",
  "district": "青秀",
  "duration": 3,
  "apartment_type": "3室",
  "time_period": "工作日",
  "mode": "ai"
}
```

响应示例

```
{
  "final_price": 197.00,
  "break_even_price": 129.66,
  "floor_price": 142.63,
  "ai_price": 140.00,
  "rule_price": 197.00,
  "gross_margin_pct": 34.1,
  "margin_factors": [
    {"factor": "供需比超高", "impact": "+20%"},
    {"factor": "青秀区域加价", "impact": "+15%"},
    {"factor": "AI模型动态调价", "impact": "x1.08倍(保本价)"},
    {"factor": "保本价安全边际", "impact": "¥130 x 1.1 = ¥143"}
  ]
}
```

六、定价建议

6.1 使用建议

- 默认使用AI定价模式：兼顾利润最大化与市场竞争力
- 纯规则模式作为后备：当AI模型不可用时自动降级
- 回南天/三月三期间关注AI输出：模型学习历史数据可能存在偏低倾向，必要时可手动微调

- 定期重新训练：建议每季度用最新订单数据重新训练，保持模型时效性

6.2 参数调整指南

参数	位置	调整影响
安全边际(1.1)	pricing_engine.py 硬编码	调高则利润更高但可能失去订单
区域加价	config.yaml districts[].premium	影响规则层定价
季节加价	config.yaml season_multipliers	影响旺季/节假日定价
供需比阈值	pricing_engine.py 第四层	调低则供需敏感度更高
目标毛利率下限	cost_parameters.csv	影响最低利润要求

6.3 后续优化方向

- 引入转化率数据：当前模型以历史成交价为训练目标，如有转化率数据可改为收益管理优化
- 强化学习替代XGBoost：当前XGBoost为静态模型，后续可用stable-baselines3的PPO/DQN实现实时动态调价
- A/B测试框架：在生产环境中同时运行AI定价和人工定价，对比实际转化率和利润

七、交付清单

文件	说明
core/feature_engineer.py	特征工程模块，从7个CSV构造25维特征
core/ml_pricing_model.py	XGBoost模型封装（训练/预测/保存/加载）
core/pricing_engine.py	四层混合定价引擎（核心）
scripts/train_model.py	模型训练脚本（R ² =0.85, MAE=0.084）
scripts/backtest.py	回测脚本（AI vs 规则 vs 旧模型）
backend/main.py	FastAPI后端（4个定价端点）
frontend/04_model_test.html	模型测试页面（AI/规则/对比模式）

文件	说明
output/models/pricing_model.json	训练好的XGBoost模型
output/figures/feature_importance.png	特征重要性图
output/backtest_results.json	回测结果数据
output/figures/backtest_comparison.png	回测对比图表

报告由 core/pricing_engine.py 自动生成，模型文件见 output/models/pricing_model.json